



PRACA DYPLOMOWA INŻYNIERSKA

Temat: Zintegrowany system do automatycznego odczytywania liczników energii elektrycznej w oparciu o technologię Modbus z wizualizacją danych w aplikacji webowej.

Dyplomant:	Grzegorz Lademann	Numer albumu: 49336
Specjalność:	Aplikacje Internetu Rzeczy	
Promotor:	dr. inż. Adam Muc	Ocena:
Recenzent		Ocena:
Data egzaminu:		Wynik studiów:

Recenzent: Promotor: Dziekan:

Wyrażam zgodę / nie wyrażam zgody na
udostępnianie mojej pracy dyplomowej

.....
data podpis

.....
Imię i Nazwisko

Gdynia, dnia r.

OŚWIADCZENIE

Świadomy/a odpowiedzialności prawnej oświadczam, że złożona praca inżynierska/magisterska* pt.: „Zintegrowany system do automatycznego odczytywania liczników energii elektrycznej w oparciu o technologię Modbus z wizualizacją danych w aplikacji webowej pracy” została napisana przeze mnie samodzielnie.

Równocześnie oświadczam, że w pracy wykorzystano tylko cytowaną literaturę, a więc praca nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz. U. 1994, nr 24, poz. 83) oraz dóbr osobistych chronionych prawem cywilnym.

Ponadto praca nie zawiera informacji i danych uzyskanych w sposób nielegalny i nie była wcześniej przedmiotem innych procedur urzędowych związanych z uzyskaniem dyplomów lub tytułów zawodowych uczelni wyższej.

Oświadczam ponadto, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną na płycie CD.

Na podstawie art. 75 §2 kodeksu postępowania administracyjnego wnoszę o odebranie tego oświadczenia jako dowodu prawdziwości okoliczności w nim podanych, przy czym jestem świadomy odpowiedzialności karnej z art. 233 §1 i §6 k.k. za złożenie fałszywego oświadczenia.

.....
podpis

* niepotrzebne skreślić

SPIS TREŚCI

Spis treści.....	3
Wykaz ważniejszych oznaczeń i skrótów.....	6
Wstęp	8
1. Przegląd technologii i rozwiązań dostępnych na rynku	9
1.1. Systemy pomiarowe w przemyśle i gospodarstwach domowych	10
1.1.1. Ewolucja urządzeń pomiarowych: od rozwiązań elektromechanicznych po Smart Metering	10
1.1.2. Współczesne podejście do monitorowania zużycia energii (sub-metering) i rola normy ISO 50001	11
1.1.3. Rola technologii IoT i protokołu Modbus w małych i średnich systemach pomiarowych.....	13
1.2. Charakterystyka protokołu Modbus w zastosowaniu automatyki.....	14
1.2.1. Warstwa fizyczna i tryby transmisji (RTU, TCP, ASCII)	15
1.2.2. Model danych, mapa rejestrów i adresowanie urządzeń.....	16
1.3. Platformy sprzętowe dla rozwiązań IoT (Raspberry Pi, Luckfox, Espressif) ..	17
1.4. Technologie wykorzystane w projekcie	19
1.4.1. Język Python i jego zastosowanie w komunikacji sprzętowej.....	19
1.4.2. Framework Django jako środowisko backendowe	20
1.4.3. Framework Bootstrap jako środowisko frontendowe	20
1.4.4. Język JavaScript i dynamiczna wizualizacja danych	20
1.4.5. Systemy bazodanowe i mapowanie obiektowo-relacyjne (ORM)	21
1.5. Mechanizmy bezpieczeństwa aplikacji internetowych.....	21
1.5.1. Protokół HTTPS i szyfrowanie transmisji	22
1.5.2. Ochrona przed atakami CSRF i mechanizmy uwierzytelniania.....	22
2. Analiza wymagań i projekt systemu „PowerTrack”	23
2.1. Wymagania funkcjonalne i poza funkcjonalne	23
2.2. Ogólna architektura systemu (model Klient-Serwer-Worker)	24
2.2.1. Warstwa sprzętowa (Worker / IoT Gateway)	25
2.2.2. Warstwa serwerowa (backend)	25
2.2.3. Warstwa kliencka (frontend).....	26
2.3. Projekt bazy danych i relacji między obiektami.....	26
2.3.1. Struktura infrastruktury sprzętowej.....	26

2.3.2. Pomiary, alerty i raportowanie	27
2.3.3. Struktura organizacyjna i użytkownicy	28
2.3.4. Struktura systemu zgłoszeń (Helpdesk).....	28
2.4. Projekt komunikacji między warstwą sprzętową, a serwerem	30
2.5. Dobór komponentów sprzętowych (liczniki, konwertery RS-485)	30
2.5.1. Liczniki energii elektrycznej	30
2.5.2. Konwerter interfejsów (UART-RS485)	31
3. Implementacja warstwy sprzętowej (IoT workers)	32
3.1. Konfiguracja środowiska uruchomieniowego na urządzeniach brzegowych ...	32
3.2. Implementacja obsługi protokołu Modbus w języku Python.....	33
3.3. Problematyka adresowania i konwersji typów danych	34
3.4. Obsługa błędów transmisji i mechanizmy retransmisji	35
3.5. Opracowanie kolejkowania danych (buforowanie RAM-dysk)	37
3.6. Mechanizm automatycznej detekcji i aktualizacji stanu urządzeń	38
3.7. Opis struktury danych i protokół wymiany informacji (JSON).....	39
3.8. Bezpieczeństwo i uwierzytelnianie urządzenia.....	39
4. Implementacja warstwy serwerowej i interfejsu użytkownika.....	41
4.1. Konfiguracja środowiska uruchomieniowego na serwerze.....	41
4.1.1. Izolacja zależności (Virtual Enviroment)	41
4.1.2. Serwer aplikacji WSGI (Gunicorn)	42
4.1.3. Generowanie lokalnego certyfikatu SSL	42
4.1.4. Odwrócony serwer proxy (Nginx).....	42
4.2. Budowa backendu aplikacji w oparciu o Django.....	43
4.2.1. Middleware i bezstanowa autentykacja urządzeń.....	43
4.2.2. Synchronizacja konfiguracji poprzez Sygnały (Signals)	44
4.3. Implementacja systemu kontroli dostępu (RBAC)	45
4.3.1. Dashboard i filtrowanie kontekstowe	45
4.4. Moduł wizualizacji danych (Frontend)	47
4.4.1. Struktura interfejsu i układ graficzny	47
4.4.2. Dashboard analityczny licznika.....	48
4.4.3. Asynchroniczne pobieranie danych (AJAX)	49
4.4.4. Mechanizm pełnoekranowy (Modal).....	50
4.4.5. Mechanizm filtrowania oraz stronicowania danych	50
4.5. Logika systemu alertowania i walidacja danych.....	51
4.6. Panel administracyjny i zarządzanie urządzeniami	53

4.7. Wystawione API dla urządzeń IoT.....	54
5. Testy i weryfikacja działania systemu	55
5.1. Środowisko testowe i metodologia badań	55
5.2. Testy integracyjne.....	55
5.3. Weryfikacja logiki biznesowej	56
5.4. Testy systemu uprawnień	57
5.5. Testy interfejsu i obsługi błędów.....	57
5.6. Ocena stopnia realizacji założeń projektowych.....	58
Podsumowanie i wnioski końcowe.....	60
Bibliografia	62
Wykaz rysunków	64
Wykaz tabel	66
Załącznik 1: Kod programu LuckFox.....	67
Załącznik 2: Kod programu Django	67
Załącznik 3: Schemat połączeń.....	67
Załącznik 4: Diagram ERD.....	67
Streszczenie	68

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

AJAX	–	<i>Asynchronous JavaScript and XML</i> Asynchroniczny JavaScript i XML
API	–	<i>Application Programming Interface</i> Aplikacyjny interfejs programistyczny
ASCII	–	<i>American Standard Code for Information Interchange</i> Amerykański Standard Kodowania Informacji (w tym tryb transmisji Modbus)
Bootstrap	–	<i>Biblioteka programistyczna (framework) CSS/JavaScript, ułatwiająca projektowania interfejsów graficznych</i>
CAPEX	–	<i>Capital Expenditures</i> Nakłady inwestycyjne ponoszone na rozwój produktu
CSRF	–	<i>Cross-Site Request Forgery</i> Fałszowanie żądań między witrynami
CSS	–	<i>Cascading Style Sheets</i> Kaskadowe arkusze stylów
DIY	–	<i>Do It Yourself</i> Idea samodzielnego wytwarzania rozwiązań
Django	–	<i>Wysokopoziomowy framework webowy napisany w języku Python, oparty na wzorcu architektonicznym MVT, wspierający szybkie tworzenie skalowalnych aplikacji internetowych</i>
EMS	–	Energy Management System System Zarządzania Energią
HMI	–	<i>Human Machine Interface</i> Interfejs Człowiek Maszyna
HTML	–	<i>HyperText Markup Language</i> Język znaczników hipertekstu
HTTPS	–	<i>HyperText Transfer Protocol Secure</i> Bezpieczny protokół przesyłania hipertekstu
IoT	–	<i>Internet of Things</i> Internet Rzeczy
IT	–	<i>Technologia informacyjna</i>
JS	–	<i>JavaScript</i> JavaScript (język programowania po stronie klienta)
ML	–	<i>Machine Learning</i> Uczenie maszynowe, obszar sztucznej inteligencji poświęcony algorytmom uczenia
Modbus	–	<i>Otwarty protokół komunikacyjny typu „master-slave”</i> wykorzystywany w automatyce przemysłowej

MVT	–	<i>Model-View-Template</i> <i>Model-Widok-Szablon</i>
MySQL	–	<i>System zarządzania relacyjnymi bazami danych działający w modelu „klient-serwer”</i>
Open Source	–	<i>Oprogramowanie o otwartym kodzie źródłowym</i>
ORM	–	<i>Object-Relational Mapping</i> Mapowanie obiektowo-relacyjne
OSD	–	<i>Operator Systemu Dystrybucyjnego</i>
OT	–	<i>Operational Technology</i> Technika operacyjna jako sprzęt i oprogramowanie służące do bezpośredniego sterowania urządzeniami fizycznymi
PDF	–	<i>Portable Document Format</i> Przenośny format dokumentów
PLC	–	<i>Programmable Logic Controller</i> Programowalny Sterownik Logiczny
Python	–	<i>Interpretowany język programowania wysokiego poziomu o dynamicznym typowaniu</i>
RBAC	–	<i>Role-Based Access Control</i> Model sterowania dostępem, w którym dostęp do zasobów różni się w zależności od roli w systemie
RTU	–	<i>Remote Terminal Unit</i> Jednostka Terminala Zdalnego (tryb transmisji Modbus)
SBC	–	<i>Single Board Computer</i> Komputer jednopłytkowy
SCADA	–	<i>Supervisory Control and Data Acquisition</i> system informatyczny nadzorujący przebieg procesu technologicznego lub produkcyjnego
SM	–	<i>Smart Metering</i> Inteligentne Opomiarowanie
SoC	–	<i>System on Chip</i> Układ scalony z systemem (System na układzie)
SSL	–	<i>Secure Sockets Layer</i> Warstwa bezpiecznych gniazd
SZE	–	<i>Systemy Zarządzania Energią</i>
SQLite	–	<i>Bez serwerowy silnik relacyjnej bazy danych</i>
TCP/IP	–	<i>Transmission Control Protocol / Internet Protocol</i> Protokół sterowania transmisją / Protokół Internetowy
WiFi	–	<i>Wireless Fidelity</i> Bezprzewodowa sieć lokalna (IEEE 802.11)
WWW	–	<i>World Wide Web</i> Sieć Światowa WWW

WSTĘP

W dobie rosnących kosztów energii elektrycznej wraz z narastającymi potrzebami przedsiębiorstw ciągle monitorowania zużycia mediów staje się kluczowym zagadnieniem zarówno dla gospodarstw domowych jak i zakładów przemysłowych. Rozwój technologii Aplikacji Internetu Rzeczy umożliwia automatyzację wielu procesów, które dotychczas wymagały czasochłonnej obsługi.

Tradycyjne liczniki energii, pomimo posiadania interfejsów komunikacyjnych takich jak RS-485 rzadko bywają zintegrowane z systemami informatycznymi. Standardowa procedura odczytu danych pomiarowych wymaga osobistej obecności przy urządzeniu pomiarowych, przy czym w takiej postaci niemożliwym jest monitorowanie w czasie rzeczywistym trendów zużycia energii szczególnie z szybkim wykrywaniem awarii.

Głównym celem niniejszej pracy było zaprojektowanie oraz implementacja systemu informatycznego „PowerTrack”, służącego do zdalnego oraz ciągłego monitorowania zużycia energii z sieci energii elektrycznej. System ma za zadanie integrować przemysłowe oraz szeroko dostępne liczniki energii elektrycznej z nowoczesną oraz intuicyjną aplikacją webową, umożliwiając tym podgląd parametrów użycia sieci w czasie zbliżonym do rzeczywistego.

W ramach projektu opracowano zintegrowaną architekturę składającą się z aplikacji webowej opartej na frameworku Django sprzężonej wraz z urządzeniami brzegowymi, na które składają się komputery jednopłytkowe umożliwiające uruchomienie systemu operacyjnego Linux. Do komunikacji z urządzeniami pomiarowymi zastosowano protokół przemysłowy Modbus RTU, a całość rozwiązania umożliwia skalowanie sieci oraz możliwość obsługi wielu liczników energii jednocześnie. W ramach pracy przygotowano również projekt warstwy sprzętowej, którego schemat ideowy znajduje się w załączniku nr. 3.

3.6. Mechanizm automatycznej detekcji i aktualizacji stanu urządzeń

Za organizację pracy całego systemu odpowiada wątek główny „MasterThread”. Jego zadaniem jest nie tylko uruchamianie podwątków (wymienionych w Rozdziale 3.5), ale także dynamiczna rekonfiguracja urządzenia na podstawie zmian zastosowanych na serwerze.

W pętli głównej „MasterThread” jest cyklicznie odpytany serwer poprzez API pozyskując znacznik czasu edycji ostatniej konfiguracji (pole „last_edit”). W przypadku wykrycia zmiany (np. użytkownik utworzył nowy licznik w panelu administracyjnym), następuje pobranie nowego pliku konfiguracyjnego JSON i „gorący restart” wątków pomiarowych bez przerywania działania samej aplikacji.

```
def _reload_config(self, json_data) -> bool:
    self._POLLING_THREAD.pause(True)
    self._FLUSHING_THREAD.pause(True)
    while self._POLLING_THREAD.is_waiting() or self._FLUSHING_THREAD.is_waiting():
        time.sleep(1)
    json_data = json_data['config']
    # [...] Tutaj następuje walidacja interwałów oraz typów danych, wyczyszczenie list
    channels = json_data.get('channels', [])
    if not channels or len(channels) == 0:
        return False
    for channel in channels:
        channel_id = channel.get('id')
        channel_name = channel.get('name')
        # [...] Pobranie pozostałych parametrów transmisji
        channel_meters = channel.get('meters', [])
        _channel = ModbusInterfaces(
            id = channel_id,
            name = channel_name,
            # [...] Pozostałe parametry transmisji
        )
        _channel.init()
        self._MODBUS_CHANNELS.append(_channel)
        for meters in channel_meters:
            # [...] Pobranie parametrów licznika oraz walidacja danych
            _meter = None
            if meter_type == "OR-WE-504":
                _meter = OR_WE_504(meter_id, _channel, meter_slave_id,
                                    meter_alert_profile)
            else:
                return False
            self._MODBUS_METERS.append(_meter)
    # -- Aktualizacja Profili -- -- #
    alert_profiles = json_data.get('alert_profiles', [])
    # [...] Weryfikacja danych profili
    for alert_profile in alert_profiles:
        _alert_profile = AlertProfile.from_json(alert_profile)
        self._ALERT_PROFILES.append(_alert_profile)
        # [...] Aktualizacja liczników w wątku próbkującym
    # [...] Odmrożenie wątków #
    return True
```

Rys. 3.7. Fragmenty implementacji metody „reload_config” zawartej w klasie „MasterThread”. Program realizuje następujące czynności: zatrzymanie wątków podrzędnych, wyczyszczenie danych, walidacja, pobranie parametrów kanałów Modbus, inicjalizacja kanałów, utworzenie odpowiednich klas liczników, odczytanie konfiguracji profili zdarzeń oraz uruchomienie pracy wątków.

Źródło: metoda „reload_config” zapisana w pliku „MasterThread.py”

4. IMPLEMENTACJA WARSTWY SERWEROWEJ I INTERFEJSU UŻYTKOWNIKA

Niniejszy rozdział stanowi szczegółowy opis implementacji centralnej części systemu „PowerTrack”. Warstwa integruje dane napływające z urządzeń IoT, zarządzając logiką biznesową, bezpieczeństwem oraz interakcją z użytkownikiem końcowym. Architektura rozwiązania została oparta na wzorcu MVT (Model-View-Template), natywnym dla frameworka Django. Niniejszy rozdział przedstawia przykładową konfigurację środowiska uruchomieniowego, szczegóły implementacji backendu aplikacji, implementację kontroli dostępu (RBAC), szczegóły implementacji modułu alertowania oraz wizualizacji danych, przedstawia fragment panelu administracyjnego oraz opisuje interfejs API wykorzystywany w komunikacji z urządzeniami brzegowymi. Z uwagi na wielką objętość kodu źródłowego, ten został zamieszczony w załączniku nr. 2 „Kod programu Django”.

4.1. *Konfiguracja środowiska uruchomieniowego na serwerze*

Jako silnik bazy danych wybrano SQLite (w fazie deweloperskiej) ze względu na brak konieczności konfiguracji zewnętrznego oprogramowania. Docelowa konfiguracja produkcyjna zakłada użycie serwera Gunicorn schowanego za serwerem proxy Nginx, który obsługuje pliki statyczne CSS/JS oraz połączenia szyfrowane SSL/TLS.

4.1.1. *Izolacja zależności (Virtual Enviroment)*

Aby uniknąć konfliktów wersji bibliotek między aplikacją a systemem operacyjnym Linux, zastosowano mechanizm wirtualizacji środowiska. W katalogu głównym projektu utworzono odseparowane środowisko przy użyciu poleceń: „python3 -m venv .venv” oraz „source .venv/bin/activate”.

Wszystkie wymagane biblioteki, zdefiniowane w pliku requirements.txt, zostały zainstalowane wewnątrz tego katalogu, co ułatwia jej aktualizację bez naruszania stabilności systemu operacyjnego hosta.

4.1.2. Serwer aplikacji WSGI (Gunicorn)

Jako interfejs między serwerem WWW a frameworkiem Django wybrano Gunicorn. Jest to serwer WSGI (Web Server Gateway Interface) klasy produkcyjnej, obsługujący wiele równoległych procesów roboczych (Workers).

Aby zapewnić ciągłość pracy, skonfigurowano Gunicorn jako usługę systemową (Systemd Service) tworząc plik konfiguracyjny „`/etc/systemd/system/PowerTrack.service`”.

4.1.3. Generowanie lokalnego certyfikatu SSL

Ze względu na przesyłanie danych uwierzytelniających, komunikacja z systemem musi być szyfrowana. W związku z tym, że urządzenie działa w sieci lokalnej bez domeny publicznej, wygenerowano certyfikat typu Self-Signed (podpisany sam przez siebie) przy użyciu narzędzia OpenSSL, do tego celu wykonano polecenie: „`sudo openssl req -x509 -nodes -days 465 -newkey rsa:2048 -keyout /etc/ssl/private/PowerTrack.key -out /etc/ssl/certs/PowerTrack.crt`”.

4.1.4. Odwrócony serwer proxy (Nginx)

Ostatnim elementem architektury jest serwer Nginx, który odpowiada za odszyfrowanie ruchu HTTPS i przekazanie czystego żądania do Gunicorn oraz serwowanie plików statycznych poprzez szybkie udostępnianie plików CSS, JavaScript.

Przykładowa serwerka HTTPS została zapisana w pliku „`/etc/nginx/sites-available/PowerTrack`” na urządzeniu pełniącym rolę serwera, przekierowując ruch HTTP na szyfrowany HTTPS, co ukazano na Rysunku 4.1.

```

server {
    listen 80;
    server_name 10.0.12.50;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name 10.0.12.50;
    ssl_certificate /etc/ssl/certs/"PowerTrack".cert;
    ssl_certificate_key /etc/ssl/private/"PowerTrack".key;
    # [...] pozostałe lokalizacje plików statycznych oraz raportów
}

```

Rys. 4.1. Przykładowa konfiguracja serwera Nginx, która zawiera sekcje „server”, a w niej port nasłuchiwania serwera HTTP oraz nazwę serwera. W drugiej sekcji „server” zawarto adres nasłuchiwania serwera HTTPS, nazwę serwera oraz ścieżki do wygenerowanego certyfikatu oraz klucza.

Źródło: opracowanie własne

4.2. Budowa backendu aplikacji w oparciu o Django

Logika aplikacji implementuje szereg mechanizmów działających w tle, które zapewniają spójność danych oraz bezpieczeństwo komunikacji API.

4.2.1. Middleware i bezstanowa autentykacja urządzeń

W standardowym podejściu Django, użytkownik uwierzytelnia się za pomocą sesji (ciasteczek). Urządzenia IoT (bramki) działają jednak w sposób bezstanowy i nie obsługują ciasteczek lub ich implementacja może być uznawana za skomplikowaną. Aby rozwiązać wspomniany problem, opracowano dedykowany komponent pośredniczący Middleware („DeviceAuthMiddleware”).

Zadaniem klasy „DeviceAuthMiddleware” jest przechwycenie każdego żądania HTTP przed dotarciem do widoku (Django View) i analiza niestandardowych nagłówek, którą zaimplementowano w metodzie „get_device_from_auth_header” (fragment programu przedstawiono na Rysunku 4.2).

```

def get_device_from_auth_header(request):
    """
    Metoda sprawdza nagłówek Authorization base64(uuid, password)
    oraz próbuje uwierzytelnić urządzenie na podstawie ww. nagłówka
    :return: Brak lub instancja zalogowanego urządzenia
    """
    auth_header = request.headers.get('Authorization')
    if not auth_header:
        return None
    try:
        if auth_header.startswith('Basic '):
            encoded_uuid_passwd = auth_header.split(' ')[1]
            decoded_uuid_passwd = base64.b64decode(encoded_uuid_passwd).decode('utf-8')

            uuid, passwd = decoded_uuid_passwd.split(':', 1)

            device = Device.authenticate(uuid=uuid, password=passwd)
            if device and device.is_active:
                return device
    except:
        return None
    return None

```

Rys. 4.2. Na załączonej fotografii przedstawiono implementację metody „get_device_from_auth_header”, która została zawarta w klasie „DeviceAuthMiddleware”. Zadaniem metody jest pobranie wymienionego w Rozdziale 3.8 nagłówka „Authorization”, weryfikacja jego treści, odkodowanie loginu oraz hasła urządzenia oraz sprawdzenie poprawności tych danych przy wykorzystaniu metody „authenticate” zawartej w klasie „Device”. Dzięki temu rozwiązaniu możliwym stało się logowanie bez użycia skomplikowanych, wbudowanych w Django mechanizmów bezpieczeństwa umożliwiając dalsze operacje przy każdorazowym żądaniu („request”).

Źródło: metoda „get_device_from_auth” zapisana w pliku „middleware.py”

4.2.2. Synchronizacja konfiguracji poprzez Sygnały (Signals)

Jednym z wymagań niefunkcjonalnych było dynamiczne przeładowanie się konfiguracji bramki IoT w przypadku zmiany jej konfiguracji na serwerze centralnym. W momencie, gdy Administrator dodaje nowy licznik z poziomu panelu administracyjnego, fizyczne urządzenie musi o tym wiedzieć. Zamiast obciążać bazę danych ciągłym odpytywaniem o pełną konfigurację zastosowano funkcje, które nasłuchują zdarzeń „post_save” (zapis rekordu) oraz post_delete (usunięcie) na modelach „Device”, „Channel” oraz „ElectricMeter”.

```

@receiver(post_save, sender=ElectricMeter)
@receiver(post_delete, sender=ElectricMeter)
def update_device_on_meter_change(sender, instance, **kwargs):
    """
        Metoda realizuje działanie, gdy zmienia się licznik, odszukuje
        powiązane urządzenie i je zapisuje co odświeża Device->last_edit_time
    """
    try:
        if instance.channel and instance.channel.device:
            device = instance.channel.device
            device.save(update_fields=['last_edit_time'])
    except Exception as ex:
        print(f"[Signals] Wystąpił błąd podczas aktualizacji czasu
        urządzenia: {ex}")

```

Rys. 4.3. Na załączonej fotografii przedstawiono sygnał Django wywoływany po zapisie lub usunięciu obiektu ElectricMeter. Funkcja wyszukuje urządzenie powiązane z licznikiem poprzez kanał i zapisuje je ponownie w celu aktualizacji pola „last_edit_time”, co pozwala oznaczyć urządzenie jako zmodyfikowane po zmianach. Dzięki temu mechanizmowi urządzenie IoT odpytując „/api/device/last_edit” otrzymuje informację o konieczności przeładowania wątków pomiarowych.

Źródło: metoda „update_device_on_meter_change” w pliku „signals.py”

4.3. Implementacja systemu kontroli dostępu (RBAC)

System „PowerTrack” obsługuje wiele grup użytkowników o rozłącznych uprawnieniach. Logika ta została zaimplementowana w warstwie widoków (Views) oraz w szablonach.

4.3.1. Dashboard i filtrowanie kontekstowe

Widok strony głównej jest przykładem dynamicznego dostosowania treści w zależności od roli zalogowanego użytkownika. Algorytm działania widoku „home” przebiega następująco:

1. pobranie wszystkich aktywnych alertów z bazy danych,
2. sprawdzenie roli użytkownika („is_user”, „is_admin”, „is_operator”),
3. filtracja danych, sprawdza czy użytkownik jest klientem końcowym oraz filtruje alerty tylko tych, które pochodzą z liczników przypisanych do jego grupy („RoomGroup”), fragment zawarto na Rysunku 4.4,
4. obliczenie liczby nieodczytanych wiadomości (np. „cnt_unread_messages”) na podstawie statusu ostatniego czasu odczytu odpowiedniego dla danej roli.

```

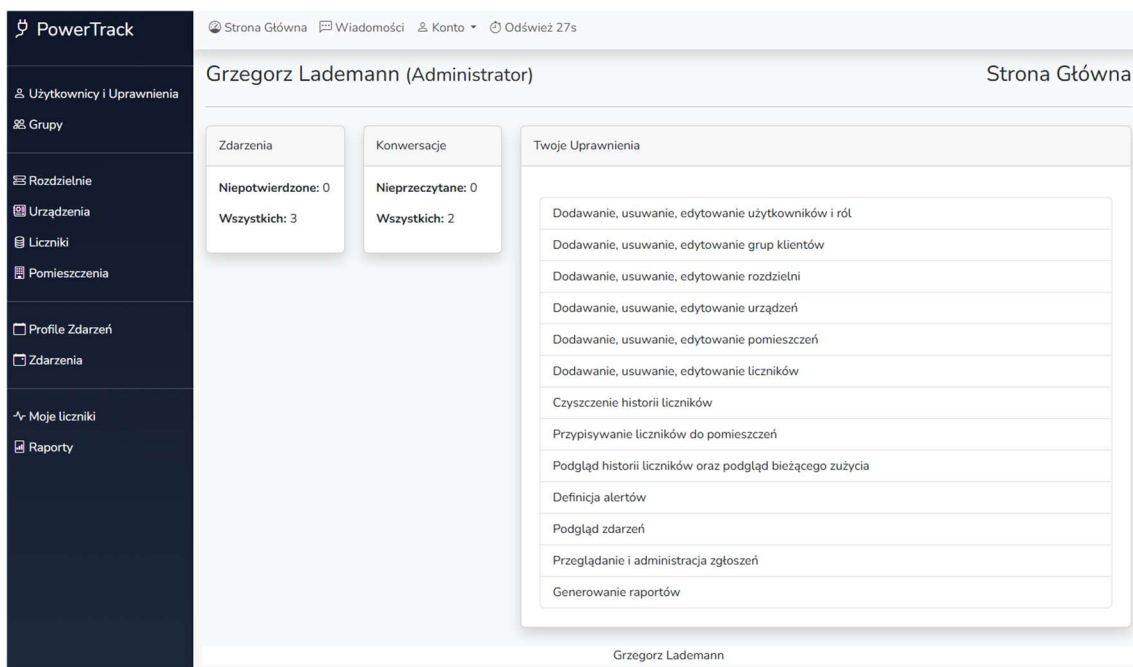
if is_user(request):
    try:
        user_group = request.user.userrole.client
    except:
        user_group = None
    finally:
        pass
    if user_group:
        alerts = alerts.filter(energy_meter__room__room_group=user_group)
    else:
        alerts = alerts.none()

```

Rys. 4.4. Rysunek przedstawia filtrację alertów dla standardowego użytkownika – użytkownik widzi tylko alerty przypisane do swojej grupy „RoomGroup”, brak przypisania skutkuje brakiem wyników.

Źródło: metoda „alerts_center” w pliku „alerts_center.py”

Efekt działania mechanizmu z podpunktu 4 zaprezentowano na Rysunku 4.5. Zalogowany użytkownik widzi zagregowane statystyki oraz listę niepotwierdzonych zdarzeń (Alertów), które wymagają jego uwagi z pominięciem danych, do których nie posiada uprawnień.



Rys. 4.5. Powitalny pulpit aplikacji z podsumowaniem liczby zdarzeń oraz konwersacji.

Źródło: Opracowanie własne

4.4. Moduł wizualizacji danych (Frontend)

Interfejs użytkownika został zaprojektowany z wykorzystaniem podejścia SPA (Single Page Application) w obrębie poszczególnych widoków analitycznych. Warstwa prezentacji opiera się na silniku szablonów Django Template Language (DTL) oraz frameworku CSS Bootstrap 5.

4.4.1. Struktura interfejsu i układ graficzny

Warstwa prezentacji w systemie „PowerTrack” została zaimplementowana w postaci widoków, gdzie każdy z nich dziedziczy z szablonu głównego, który definiuje siatkę strony. Układ graficzny podzielono na kluczowe strefy funkcjonalne:

1. Panel boczny: jest elementem nawigacyjnym („side_panel.html”), ładowany dynamicznie za pomocą tagu „{% include %}”, a jego zawartość zmienia się w zależności od zalogowanego użytkownika. Dla administratora wyświetlone zostaje pełne menu konfiguracyjne (użytkownicy, infrastruktura, alerty). Użytkownicy mają ograniczoną widoczność wyłącznie do zakładek „Moje Liczniki”, „Raporty” oraz „Zdarzenia”. Panel ten pozwala na nawigację w zaimplementowanym systemie „PowerTrack”, ukazany w lewej części na granatowym tle na rysunku 4.5.
2. Panel górny: zawiera informacje dotyczącą ostatniego czasu odświeżenia oraz skróty nawigacyjne do modułu „Wiadomości”, „Konto” czy „Strona Główna”. Panel ten został zaprojektowany z myślą o użytkowniku, gdzie może uzyskać dostęp do zaimplementowanego systemu zgłoszeń, zmiany hasła, wylogowania czy przeglądu szczegółów konta, ukazany na górnym pasku nawigacyjnym.
3. Obszar roboczy: to dynamiczny blok „{% block main%}”, w którego miejsce jest wstawiana właściwa treść dla danego widoku (np. ustawienia, wykresy, tabele). Dzięki zastosowaniu kontenera „container-fluid” obszar ten w całości wypełnia dostępną szerokość ekranu, ukazany w prawym dolnym rogu na białym tle (Rysunek 4.5).
4. System komunikatów zwrotnych (Toasts): w aplikacji webowej zaimplementowano interakcję z użytkownikiem poprzez system wyskakujących powiadomień. W tym celu wykorzystano wbudowany w Django framework „messages”, umożliwiający przekazywanie komunikatów z backendu (np. „Zapisano pomyślnie”, „Niepoprawne hasło”) do frontendu. Po stronie przeglądarki, skrypt JavaScript automatycznie

przechwytuje te komunikaty, które zostają wyświetlone w prawej dolnej części ekranu.

4.4.2. Dashboard analityczny licznika

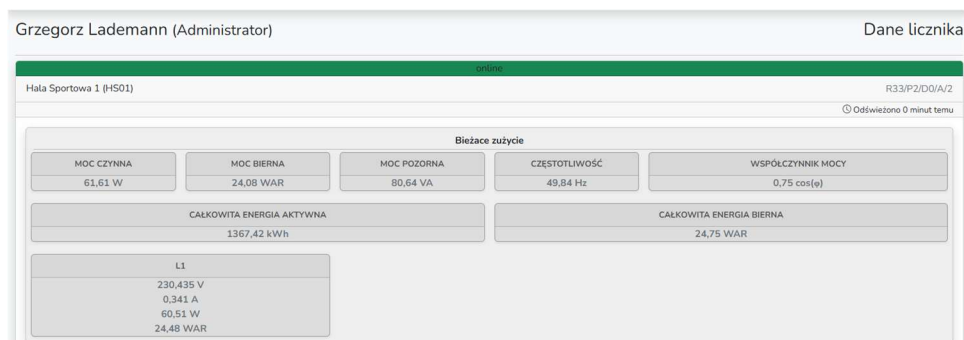
Centralnym punktem interfejsu jest widok licznika („my_meters_details”), który agreguje dane w trzech sekcjach:

1. sekcja statusu bieżącego wyświetla ostatnie odczyty licznika (moc czynna, bierna, napięcie) w formie kafelków (implementację kafelka wyświetlającego moc czynną zawarto na Rysunku 4.6). Zastosowano warunkowe formatowanie stylów, w przypadku, gdy dane licznika będą zbyt stare to wartości automatycznie są przekreślane, a nagłówek zmienia kolor na ostrzegawczy, co pozwala na szybką identyfikację problemów. Wizualną reprezentację tych danych przedstawiono na Rysunku 4.7. Sekcja ta pozwala użytkownikowi systemu na szybką ocenę parametrów bieżących bez konieczności analizy wykresów,
2. sekcja filtrów czasowych umożliwia wybór zakresu dat z walidacją po stronie klienta,
3. sekcja wykresów dynamicznych wykorzystuje bibliotekę Chart.js do renderowania interaktywnych przebiegów czasowych na elemencie HTML5 “<canvas>”.

```
<div class="col-2">
  <div class="card h-100 shadow-sm" style="background-color: #ddd; border:
    1px solid #999">
    <div class="card-header">
      <p class="fw-bold small m-0 text-muted text-uppercase">Moc czynna</p>
    </div>
    <div class="card-body py-0">
      <p class="fw-bold text-secondary m-0 {% if meter.is_timed_out %}text-
        decoration-line-through{%endif%}">{{ meter.last_measurement.total_
          active_power }} W</p>
    </div>
  </div>
</div>
```

Rys. 4.6. Fragment kodu HTML odpowiedzialny za prezentację aktualnej wartości mocy czynnej „total_active_power”. W przypadku przekroczenia dopuszczalnego czasu od ostatniego pomiaru „is_timed_out” wartość jest wizualnie oznaczana poprzez przekreślenie, co jednoznacznie wskazuje na nieaktualność danych.

Źródło: fragment pliku „my_meter_details.html”



Rys. 4.7. Panel parametrów bieżących licznika – widok kart statusowych.

Źródło: Opracowanie własne

4.4.3. Asynchroniczne pobieranie danych (AJAX)

Aby uniknąć przeładowania całej strony przy zmianie zakresu dat, zaimplementowano mechanizm asynchroniczny z wykorzystaniem Fetch API. Funkcja „fetchMeterData()” wysyła zapytanie GET do endpointu „/metres/{id}/chart_data”, pobierając jedynie surowe dane w formacie JSON. Następnie funkcja „updateCharts(data)” przetwarza otrzymaną odpowiedź oraz ponownie przerysowuje obiekty wykresów. Pozwala to na redukcję transferu danych sieciowych oraz ulepszenie wrażeń użytkownika. W ten sposób zostały wygenerowane przebiegi czasowe, prezentujące zmienność obciążenia oraz parametrów jakościowych energii, ukazano na Rysunku 4.8.

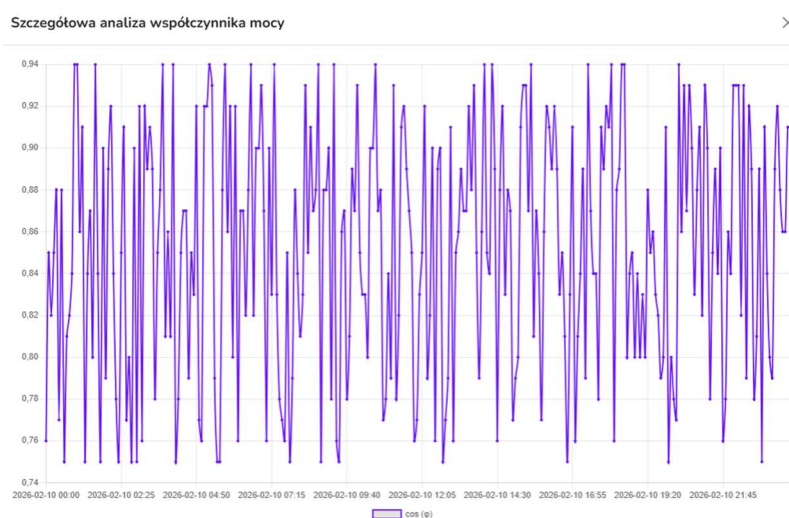


Rys. 4.8. Panel parametrów bieżących licznika – widok kart statusowych.

Źródło: Opracowanie własne

4.4.4. Mechanizm pełnoekranowy (Modal)

Ze względu na dużą ilość danych na wykresach (np. przebiegi 3-fazowe), zaimplementowano funkcję „openFullscreen()”. Po kliknięciu odpowiedniego przycisku, skrypt JS kopiuje konfigurację wybranego wykresu oraz wyświetla go w oknie modalnym (Bootstrap Modal) o zwiększonych wymiarach, co ułatwia szczegółową analizę anomalii. Przykład zastosowania wyskakującego układu graficznego przedstawiono na rysunku 4.9.



Rys. 4.9. Okno modalne przedstawiające powiększony wykres współczynnika mocy względem czasu.

Źródło: Opracowanie własne

4.4.5. Mechanizm filtrowania oraz stronicowania danych

W systemach klasy przemysłowej liczba rekordów (np. zdarzeń historycznych) może szybko osiągnąć wartości rzędu tysięcy. Aby zapewnić płynność działania interfejsu oraz zwiększyć czytelność danych, w widokach zawierających tabele zaimplementowano dwa mechanizmy:

1. system umożliwia użytkownikowi zawężenie wyników wyszukiwania za pomocą filtrów (np. po nazwie, grupie, rozdzielni). Parametry te są przekazywane do serwera za pomocą metody GET w adresie URL. Po stronie backendu następuje nakładanie filtrów na obiekt „QuerySet”, dzięki czemu zapytanie SQL do bazy danych jest zoptymalizowane oraz zwraca rekordy spełniające zadane kryteria,

```

fname = request.GET.get('fname')
fis_active = request.GET.get('fis_active')
fphases = request.GET.get('fphases')
alert_profiles = AlertProfile.objects.all()
# -- -- #
if fname:
    alert_profiles = alert_profiles.filter(name__icontains=fname)
if fis_active:
    alert_profiles = alert_profiles.filter(is_active= ( fis_active == "Tak" ) )
if fphases:
    if "1" in fphases:
        alert_profiles = alert_profiles.filter(l1_enable=True)
    if "2" in fphases:
        alert_profiles = alert_profiles.filter(l2_enable=True)
    if "3" in fphases:
        alert_profiles = alert_profiles.filter(l3_enable=True)

```

Rys. 4.10. Na załączonym fragmencie przedstawiono mechanizm dynamicznego filtrowania profili alarmowych na podstawie zapytania HTTP (GET). System pobiera parametry filtrujące z zapytania użytkownika (fname, fis_active), a następnie buduje zapytanie do bazy danych w sposób warunkowy.

Źródło: metoda „manage_alerts” w pliku „alerts.py”

- przefiltrowane wyniki nie tylko są wyświetlane w całości, lecz są przekazywane do klasy „Paginator”, która dzieli wyniki na strony o stałej wielkości (skonfigurowano 25 rekordów na stronę). Implementacja tego mechanizmu zawarto na rysunku 4.11. Mechanizm ten drastycznie redukuje czas generowania widoku HTML oraz obciążenie łącza internetowego, przesyłając do klienta jedynie fragment danych.

```

# -- -- Paginacja -- -- #
paginator = Paginator(alert_profiles, 25)
page_number = request.GET.get('page')
page_object = paginator.get_page(page_number)
params = request.GET.copy()
if "page" in params:
    del params["page"]
# -- -- -- -- -- #

```

Rys. 4.11. Na załączonym fragmencie przedstawiono mechanizm paginacji, który dzieli przefiltrowane wyniki na strony. W odczytanym fragmencie odczytywany jest numer strony „page” oraz konstruowany jest „page_object” przechowujący listę obiektów znajdujących się na danej stronie.

Źródło: metoda „manage_alerts” w pliku „alerts.py”

4.5. Logika systemu alertowania i walidacja danych

Moduł zarządzania zdarzeniami (alerts.py) jest kluczowym elementem systemu, odpowiadającym za definicję progów bezpieczeństwa. Z uwagi na charakter monitorowanych wielkości, formularze edycyjnej („manage_edit_alert”) zawierają rozbudowaną logikę walidacji, zapobiegającą wprowadzaniu ustawień sprzecznych. Formularz edycji ukazano na rysunku 4.12.

Zaimplementowano weryfikację (Rysunek 4.13), która blokuje zapisanie profilu, jeśli:

1. Wartość minimalna jest większa od maksymalnej (np. „min_voltage” > „max_voltage”),
2. Wartości wykraczają poza fizyczne możliwości liczników (np. ujemna częstotliwość)
3. Wykryto duplikat nazwy profilu.

Nazwa	Aktywny	Minimum	Maksimum	Jednostka
Napięcie	☒	229	230	[V]
Prąd	☒	0	80	[A]
Częstotliwość	☒	49,5	50,5	[Hz]
Moc czynna	☒	-18400	18400	[W]
Moc bierna	☐	-18400	18400	[W]
Jakość	☒	0.2	1.0	[cosφ]

Rys. 4.12. Modal przedstawiający formularz edycji profilu zdarzeń. W oknie wyszczególniono nazwę edytowanego „Profilu Zdarzeń”, jego nazwę, stan aktywności, wybór faz, które ma monitorować oraz pozostałe parametry wraz z flagami włączenia oraz przedziałami wartości dla napięć, prądów itp.

Źródło: Opracowanie własne

```
if (fmin_voltage>fmax_voltage or fmin_current>fmax_current or
    fmin_frequency>fmax_frequency or
    fmin_active_power>fmax_active_power or
    fmin_reactive_power>fmax_reactive_power or
    fmin_power_factor>fmax_power_factor):
    message_error(request, __TITLE__, "Minimum nie może być większe od
    maksimum.")
    return redirect(__REDIRECT__)

# [...] Dalsza część kodu
alert_profile = AlertProfile.objects.create(...)
```

Rys. 4.13. Modal przedstawiający fragment weryfikacji formularza, który dokonuje walidacji spójności logicznej progów alarmowych w widoku kreowania profilu.

Źródło: metoda „manage_add_alert” w pliku „alerts.py”

4.6. Panel administracyjny i zarządzanie urządzeniami

W niniejszym projekcie „PowerTrack” stworzono dedykowany interfejs webowy, zastępujący standardowy, hermetyczny panel Django Admin. Panel ten został podzielony na funkcjonalne sekcje tematyczne:

- użytkownicy i klienci: zarządzanie dostępem osobowym,
- infrastruktura: definiowanie rozdzielni (SwitchBoard), bramek IoT (Devices), liczników oraz pomieszczeń,
- diagnostyka: definiowanie profili zdarzeń (AlertProfile) oraz przegląd zdarzeń
- rozliczenia i historia: przegląd odczytów w czasie rzeczywistym, przegląd historii oraz generowanie oraz wydruk raportów.

Wszystkie formulacje edycyjne zabezpieczono tokenami CSRF, a operacje usuwania kluczowych elementów (np. liczników) wymagają dodatkowego potwierdzenia, aby zapobiec przypadkowej utracie historii pomiarowej.

Widok „manage_devices” („Zarządzanie Urządzeniami”) przedstawia listą bramek IoT wraz z informacją, kiedy poszczególny był ostatnio aktywny, ukazano na rysunku 4.14.

UUID	Aktywny	Rozdzielnia	Adres IP	Typ	Ostatnio widziany	Data instalacji	Opis	Akcja
D0	Tak	R33/P2	10.0.13.10	LP_MAX	11 lutego 2026	16 listopada 2025	Urządzenie znajdujące się w rozdzielni R33	[ikony]

Rys. 4.14. Widok „Zarządzanie Urządzeniami” przedstawiający panel zarządzania urządzeniami wraz z wyświetleniem daty ostatniego połączenia.

Źródło: Opracowanie własne

4.7. *Wystawione API dla urządzeń IoT*

System udostępnia zestaw endpointów REST API (zdefiniowanych w pliku `urls.py`), które służą do komunikacji z warstwą sprzętową. Endpointy zwracają wyniki w formacie JSON (`JsonResponse`) i są opisane wcześniej wspomnianym middlewarem.

Najważniejszymi z nimi są:

- „`/api/ping`”: służy do monitorowania dostępności serwera,
- „`/api/device/config`”: zwraca strukturę konfiguracyjną (kanały, liczniki, profile) dla danego urządzenia,
- „`/api/device/send_measurements`”: endpoint przyjmuje tablicę obiektów pomiarowych (metoda POST),
- „`/api/device/send_alert`”: odbiera obiekty zawierające informacje o anomaliach związanych z profilami zdarzeń,
- „`/api/device/last_edit`”: prosty endpoint zwracający jedynie ostatni czas edycji urządzenia w postaci timestamp.

5. TESTY I WERYFIKACJA DZIAŁANIA SYSTEMU

Niniejszy rozdział przedstawia proces weryfikacji oprogramowania systemu „PowerTrack”. Skoncentrowano się na poprawności działania logiki biznesowej, mechanizmach przetwarzania danych (backend) oraz interakcji użytkownika z interfejsem graficznym (Frontend). Pominięto weryfikację metrologiczną samych liczników, tym samym skupiając się na warstwie integracyjnej i aplikacyjnej. W dalszej części pracy zostało omówione środowisko testowe, rezultaty testów integracyjnych, weryfikacji logiki biznesowej, testy systemu uprawnień, poprawność obsługi błędów oraz ocena stopnia realizacji założeń projektowych zawartych w rozdziale 2.

5.1. Środowisko testowe i metodologia badań

Badania przeprowadzono metodą czarnej skrzynki, weryfikując zachowanie systemu na zadane wymuszenia bez ingerencji w kod źródłowy w trakcie testu. Środowisko testowe składało się z następujących elementów:

- serwer aplikacji uruchomiony w środowisku produkcyjnym,
- baza danych, która została wypełniona danymi testowymi,
- klient, przeglądarka Opera (Desktop),
- urządzenie brzegowe LuckFox Lyra Plus,
- router sieciowy Mikrotik hap ac2,
- licznik Orno OR-WE-504,
- licznik Orno OR-WE-516.

5.2. Testy integracyjne

Celem testów było potwierdzenie, że dane odczytane przez procesy uruchomione na urządzeniu brzegowym są poprawnie buforowane, przesyłane i zapisywane w centralnej bazie danych.

Tabela 5.1

Scenariusze testowe integracji Worker-Backend

Lp.	Nazwa testu	Warunki początkowe	Wykonana akcja	Oczekiwany rezultat	Wynik
L1	Stabilność komunikacji	System uruchomiony, podłączony licznik	Obserwacja wydruku programu na urządzeniu brzegowym wraz z obserwacją kolejki przez 30 minut	W kolejce „RamdiskQueue” zostanie zarejestrowane 12 odczytów (interwał 5 min)	Pozytywny
L2	<i>Odporność na błędy komunikacji w warstwie sprzętowej</i>	System uruchomiony, licznik podłączony	Fizyczne odłączenie licznika na 10 minut i ponowne połączenie	Proces wykrył błąd (pominięcie odczytu). Po przywróceniu połączenia komunikacja została wznowiona automatycznie	Pozytywny
L3	<i>Odporność na błędy komunikacji sieciowej</i>	System uruchomiony, router odłączony od urządzenia brzegowego	Praca systemu bez dostępu do sieci LAN przez 15 minut, następnie podłączenie łącza	Urządzenie brzegowe gromadzi dane w buforze RAM. Po uzyskaniu łącza nastąpiło wysyłanie danych do serwera	Pozytywny

Źródło: opracowanie własne

5.3. Weryfikacja logiki biznesowej

Najważniejszymi składnikami systemu z finansowego punktu widzenia jest nie tylko moduł powiadomień, a także możliwość generowania raportów zużycia z intuicyjnym mechanizmem zapisywania tych danych w założonym formacie.

Tabela 5.2

Scenariusze weryfikacji logiki raportowania i alertowania

Lp.	Nazwa testu	Warunki początkowe	Wykonana akcja	Oczekiwany rezultat	Wynik
A1	Detekcja przekroczeń	Do licznika przypisano próg alarmowy dla prądu powyżej 1A	Do licznika podłączono obciążenie w postaci grzejnika elektrycznego 1000W z ustawieniem na pełną moc	W wątku „PollingThread” następuje wykrycie przekroczenia. W bazie danych zostaje utworzony wpis „Alert” ze statusem niepotwierdzony	Pozytywny
A2	<i>Pobranie raportu ze zdarzenia w formacie PDF</i>	Wykorzystanie wykreowanego zdarzenia z Scenariusza A1	Odnalezienie zdarzenia, kliknięcie przyciskiem ikony drukarki przy zdarzeniu, po którym następuje otwarcie nowej karty przeglądarki, wybranie opcji „Drukuj”	Zapisanie na komputerze pliku PDF	Pozytywny
A3	<i>Wygenerowanie i pobranie raportu w formacie CSV dotyczącego licznika energii elektrycznej</i>	System uruchomiony przez 24h	Wygenerowanie raportu w sekcji „Raporty” dla wybranego zakresu dat	Pobrano plik CSV zawierające poprawne kolumny (czas, energia czynna, bierna) zgodne z danymi w bazie	Pozytywny

Źródło: opracowanie własne

5.4. Testy systemu uprawnień

Zweryfikowano poprawność działania systemu kontroli dostępu (RBAC), celem upewnienia, że użytkownicy końcowi nie mają dostępu do danych wrażliwych ani możliwości rekonfiguracji systemu.

Tabela 5.3

Wyniki testów bezpieczeństwa aplikacji webowej

Lp.	Nazwa testu	Próba dostępu	Oczekiwany rezultat	Wynik
S1	<i>Blokada niezalogowanych użytkowników</i>	Wejście na „/metres/list” (Zarządzanie Licznikami” jako niezalogowany użytkownik	Sesja przeglądarki zostanie przekierowana na zasób „/account/login”	Pozytywny
S2	<i>Blokada nieautoryzowanego dostępu nr.1</i>	Wprowadzenie adresu „/metres/list” jako użytkownik niemający praw administracyjnych	Przekierowanie sesji przeglądarki na zasób „/error_view” z komunikatem „Nie masz dostępu do tego zasobu.”	Pozytywny
S3	<i>Blokada nieautoryzowanego dostępu nr.2</i>	Wprowadzenie adresu „/manage/metres/delete/2” (Usunięcie licznika o id 2) jako użytkownik typu operator	Odmowa dostępu z komunikatem „Nie masz dostępu do tego zasobu.” Z uwagi na to, że operator może tylko przeglądać dodawać i edytować	Pozytywny
S4	<i>Edycja krytycznych ustawień</i>	Edytowanie użytkownika systemu „/manage/users/edit/1” poprzez zmianę hasła	Dostęp zostaje przyznany, poprawnie udaje się zmienić hasło użytkownika, potwierdzając to komunikatem.	Pozytywny

Źródło: opracowanie własne

5.5. Testy interfejsu i obsługi błędów

Ostatnim etapem była weryfikacja aplikacji w sytuacjach awaryjnych oraz odporności interfejsu na błędne działania użytkownika. Skupiono się na mechanizmach informowania o statusie infrastruktury oraz walidacji danych wejściowych w przeglądarce.

Tabela 5.4

Wyniki testów warstwy prezentacji

Lp.	Nazwa testu	Obserwacja	Wniosek
I1	Wizualizacja „Status Offline”	Po upływie dwukrotności najdłuższego czasu pobierania lub wysyłania odczytów status licznika w panelu „Moje liczniki” nagłówek zmienił kolor z zielonego „Online” na żółty „Offline”	Interfejs poprawnie informuje o awarii infrastruktury
I2	Obsługa nieistniejących zasobów	Wpisanie w pasku adresu niepoprawnej ścieżki „./metres/usuń”. System wyświetla komunikat o nieznanym zasobie z kodem błędu 404.	Aplikacja jest zabezpieczona przed błędami nawigacji
I3	Walidacja formularzy	Próba wprowadzenia adresu Modbus dla licznika spoza dopuszczalnego zakresu 1-247 skutkuje komunikatem po stronie frontendu „Wartość nie może być większa niż 247” co uniemożliwia dalsze wysłanie formularza.	Mechanizm walidacji skutecznie chroni bazę danych przed wprowadzeniem nieprawidłowych danych
I4	Wyświetlanie wykresów	Otwarcie widoku „Dane licznika” skutkuje wyświetleniem danych bieżących, czasem ostatniego odczytu oraz wykresów dla przebiegów napięcia, prądu, mocy oraz energii	Aplikacja webowa poprawnie wyświetla dane pomiarowe w czasie

Źródło: opracowanie własne

5.6. Ocena stopnia realizacji założeń projektowych

Na podstawie przeprowadzonych testów funkcjonalnych oraz analizy wytworzonego oprogramowania dokonano weryfikacji stopnia spełnienia wymagań zdefiniowanych w początkowej fazie projektu.

Tabela 5.5

Weryfikacja wymagań funkcjonalnych systemu „PowerTrack”

Lp.	Treść wymagania	Sposób realizacji / Dowód	Status
1	Akwizycja danych (Modbus RTU)	Zaimplementowano klasę „OrmoModbusClient”, klasy „OR_WE_504”, „OR_WE_516” przechowujące adresy rejestrów oraz wątek „PollingThread” (Rozdział 3). Testy L1 potwierdziły cykliczny odczyt rejestrów.	Spełnione
2	Obsługa wielu urządzeń	System obsługuje dynamiczne dodawanie urządzeń oraz liczników w bazie. Testy przeprowadzono na dwóch modelach Ormo (OR-WE-504 oraz OR-WE-516)	Spełnione
3	Wizualizacja danych (Wykresy)	Zaimplementowano widok z biblioteką Chart.js (Rozdział 4). Testy I4 potwierdziły generowanie wykresów liniowych	Spełnione
4	System alarmowania	Stworzono mechanizm „AlertProfile” i walidację progów w wątku „PollingThread”. Test A1 wykazał poprawne wykrycie przeciążenia	Spełnione
5	Archiwizacja danych	Wykorzystano silnik SQLite/Django ORM. Wszystkie odczyty są trwale zapisane w tabeli „MeterReading”	Spełnione
6	Zarządzanie konfiguracją	Zbudowano dedykowany panel administracyjny „/manage” umożliwiający pełną edycję infrastruktury (CRUD)	Spełnione

Źródło: opracowanie własne

Tabela 5.6

Weryfikacja wymagań niefunkcjonalnych systemu „PowerTrack”

Lp.	Treść wymagania	Sposób realizacji / Dowód
1	<i>Czytelność interfejsu (Full HD)</i>	Interfejs oparty o Bootstrap 5 poprawnie skaluje się na ekranach Desktop (testowano na przeglądarce Opera)
2	<i>Stabilność pracy</i>	Zaimplementowano mechanizm Watchdog oraz obsługę wyjątków „try-except”. Testy L2 wykazały, że odłączenie licznika nie zawiesza działania systemu
3	<i>Bezpieczeństwo</i>	System ról (Admin/Operator/Użytkownik) oraz middleware autoryzacji. Testy S1-S4 potwierdziły brak dostępu dla osób niepowołanych
4	<i>Otwartość architektury</i>	System zbudowano modułowo (osobno Worker, osobno Web). Użycie frameworka Django pozwala na łatwe rozbudowanie istniejącej aplikacji

Źródło: opracowanie własne

PODSUMOWANIE I WNIOSKI KOŃCOWE

Celem niniejszej pracy było zaprojektowanie oraz implementacja kompleksowego systemu monitoringu paramentów sieci energetycznej o nazwie „PowerTrack”. W ramach projektu zintegrowano warstwę automatyki przemysłowej (OT) z systemami IT, gdzie zrealizowano w pełni funkcjonalne rozwiązanie klasy IoT, integrujące warstwę sprzętową (opartą o komunikację w standardzie Modbus RTU) z nowoczesną warstwą serwerową wykonaną w technologii Django.

Głównym założeniem było stworzenie narzędzia informatycznego, które umożliwi zdalną akwizycję, archiwizację oraz wizualizację danych pomiarowych. Przeprowadzone w Rozdziale 5 testy weryfikacyjne w jednoznaczny sposób potwierdziły, że główny cel pracy został w pełni osiągnięty, a powstały system spełnia wszystkie zdefiniowane w Rozdziale 2 założenia funkcjonalne i нефункционалне.

Realizacja złożonego projektu wymagała doboru odpowiednich rozwiązań technologicznych, zapewniających stabilność oraz skalowalność systemu. W pracy wykorzystano następujące środowiska i narzędzia:

- język programowania Python oraz framework Django 6.02 stanowiły fundament logiki biznesowej. Wybór ten został podyktowany bogatą biblioteką modułów wraz z wbudowanym systemem ORM, co znacząco przyspieszyło proces tworzenia bazy danych,
- HTML, CSS oraz framework Bootstrap 5 posłużyły do budowy interfejsu użytkownika,
- Pycharm 2024.2.2 (Community Editon) został wykorzystany jako główne środowisko programistyczne (IDE), wspierając proces debugowania kodu,
- Ubuntu Server został wykorzystany jako system operacyjny serwera oraz urządzenia brzegowego, co zapewniło stabilne środowisko uruchomieniowe dla usług Gunicorn, Nginx oraz środowisk Python,
- OpenSSL wykorzystano do wygenerowania certyfikatów bezpieczeństwa, niezbędnych do szyfrowania transmisji danych.

W rozdziałach 3 i 4 szczegółowo opisano proces konfiguracji warstwy klienckiej (urządzenie brzegowe) oraz serwerowej. Największym wyzwaniem okazało się zapewnienie ciągłości akwizycji danych przy jednoczesnej obsłudze zapytań użytkownika. Problem rozwiązano poprzez zastosowanie architektury wielowątkowej w warstwie sprzętowej oraz asynchronicznych zapytań AJAX w warstwie prezentacji.

Zaprojektowany mechanizm buforowania danych (opisany w Rozdziale 3) skutecznie wyeliminował problem utraty danych pomiarowych w momentach awarii łącza internetowego, co jest kluczowe dla systemów podobnej klasy.

Otwarta i modułarna architektura systemu „PowerTrack” pozwala na jego łatwą rozbudowę w przyszłości, gdzie potencjalne kierunki rozwoju obejmują:

1. moduł predykcji zużycia, który byłby oparty na algorytmach uczenia maszynowego (ML) w celu prognozowania przyszłego zapotrzebowania na energię na podstawie danych historycznych,
2. rozwinięcie warstwy prezentacji aplikacji o możliwość poprawnego skalowania się treści na urządzeniach mobilnych,
3. implementacja modułu rozliczeń finansowych i obsługi taryf umożliwiłaby na zautomatyzowanie procesu rozliczeń z najemcami oraz na precyzyjniejszą alokację kosztów zużycia, gdyż stałoby się możliwym określenie stawki za kWh w zależności od pory dnia,
4. rozszerzenie obsługi większej ilości modeli liczników energii elektrycznej, przedstawiony projekt mógłby zostać rozwinięty o moduł dynamicznego ładowania sterowników dla nowych modeli liczników.

Wykonany projekt stanowi kompletne oraz działające narzędzie, które z powodzeniem może zostać wdrożone w rzeczywistych obiektach, takich jak budynki najmu, małe zakłady produkcyjne, biurowce czy inteligentne domy, przyczyniając się do optymalizacji zużycia energii elektrycznej.

STRESZCZENIE

Dyplomant: Grzegorz Lademann

Promotor: dr inż. Adam Muc

Tytuł pracy: Zintegrowany system do automatycznego odczytywania liczników energii elektrycznej w oparciu o technologię Modbus z wizualizacją danych w aplikacji webowej.

Niniejsza praca dyplomowa stanowi opracowanie oraz implementację systemu monitoringu energii elektrycznej, której celem jest wspomaganie optymalizacji energetycznej obiektów przemysłowych oraz gospodarstw domowych. Projekt stanowi odpowiedź na rosnące koszty mediów oraz rygorystyczne wymagania dyrektyw unijnych, oferując alternatywę dla kosztownych systemów komercyjnych.

Głównym założeniem było stworzenie autorskiego systemu „PowerTrack”, który jest budżetowym, ale w pełni funkcjonalnym rozwiązaniem klasy Internetu Rzeczy (IoT). Architektura systemu integruje warstwę sprzętową, opartą o liczniki komunikujące się w standardzie Modbus RTU z nowoczesną warstwą serwerową wykonaną w języku Python i frameworku Django. Kluczowym osiągnięciem była implementacja wielowątkowego mechanizmu, rozdzielającego warstwę fizyczną od warstwy aplikacyjnej, dzięki czemu rozwiązanie stało się odporne na błędy komunikacji, awarie łącza internetowego gwarantując ciągłość pomiarów po przywróceniu połączenia.

Aplikacja udostępnia użytkownikowi czytelny panel sterowania w przeglądarce, umożliwiając bieżącą analizę parametrów takich jak napięcie, prąd czy moc czynna. System automatycznie wykrywa anomalie oraz umożliwia generowanie raportów zużycia, umożliwia komunikację z operatorem wewnątrz ekosystemu, czy przeglądanie przekroczeń. Przeprowadzone testy w warunkach rzeczywistych potwierdziły stabilność działania oraz bezpieczeństwo danych, co pozwoliło na osiągnięcie celu pracy dostarczając gotowego rozwiązania wspierającego efektywne zarządzanie energią.

Słowa kluczowe:

Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki inżynierskie i techniczne, Elektronika, inżynieria informatyczna, Internet Rzeczy (IoT), monitoring energii, aplikacja webowa, Modbus RTU

ABSTRACT

Author: Grzegorz Lademann

Supervisor: dr eng. Adam Muc

Thesis title: Integrated system for automated electricity meter reading based on Modbus technology with data visualization in web application.

This diploma thesis presents the design and implementation of an electrical energy monitoring system aimed at supporting energy optimization in industrial facilities and households. The project addresses rising utility costs and stringent EU directive requirements by offering a cost-effective alternative to expensive commercial systems.

The main objective was to create the proprietary “PowerTrack” system, a budget-friendly yet fully functional Internet of Things (IoT) solution. The system architecture integrates a hardware layer, based on meters communicating via the Modbus RTU standard, with a modern server layer developed using Python and the Django framework. A key achievement was the implementation of multi-threaded mechanism that separates the physical layer from the application layer. This ensures the solution is resilient to communication errors and internet connection failures, guaranteeing measurement continuity upon reconnection.

The application provides users with a clear, browser-based dashboard, enabling real-time analysis of parameters such as voltage, current and active power. The system automatically detects anomalies, generates consumption reports, facilitates communication with the operator within the ecosystem, and allows for the review of threshold exceedances. Tests conducted in real-world conditions confirmed operational stability and data security. The thesis objective was achieved by delivering a ready-to-use solution that supports effective energy management.

Key words:

Fields of science and technology according to OECD requirements: Engineering and technology, Electronic engineering, information engineering, Internet of Things (IoT), energy monitoring, web application, Modbus RTU